Reproducibility and Release Evidence: Regenerating the Artifact Chain

Reproducibility and Release Evidence: Regenerating the Artifact Chain I

Every artifact in this work is regeneratable from source with fixed seeds; nothing in `output/` is authoritative, and nothing learner-facing is hand-typed.

Deterministic kernel outputs, stable graph ordering, and rerunnable counts

Deterministic kernel outputs, stable graph ordering, and rerunnable counts I

The kernels use natural-log units and either no randomness or an explicitly seeded NumPy generator; the worked examples register rounded values so two runs of `build_claim_registry()` are byte-identical. The curriculum graph's topological order processes ready nodes in sorted-id order, so the exported SkillTree JSON is stable across runs.

Verification gates for code, prose, artifacts, and rendered surfaces

Verification gates for code, prose, artifacts, and rendered surfaces I

The gate tables treat the curriculum build as scientific computing: reported values should be traceable to workflow steps, custom code should be tested and versioned, and research claims should be reported with enough transparency that readers can distinguish reproducible local artifacts from future replication or learner-outcome evidence (Sandve et al. 2013; Wilson et al. 2014; Munafò et al. 2017).

The gates split into two families. The first runs anywhere the repository can be checked out, with no Docker daemon and no browser.

Verification gates for code, prose, artifacts, and rendered surfaces II

Offline gate	Command	Evidence
Tests + coverage	`uv run pytest --cov=src --cov-fail-under=90`	enforced aggregate floor: 90%; use command output as the current coverage number, and read the module caveat below
Kernel coverage	within the coverage run	kernel package coverage is checked inside the same suite rather than asserted from prose
Lint	`uvx ruff check src scripts tests`	clean required
Format	`uvx ruff format --check src scripts tests`	clean required
Types	`uv run mypy src`	no issues required
No mocks	`rg "unittest|MagicMock|@patch|autospec" tests/`	no mock factories should be present
Provenance	`scripts/run_kernels.py` and `scripts/export_skilltree.py`	every claim token resolves and the bare-number linter reports zero violations
Scholarship	every authored citation maps to a BibTeX entry and source-role lane	enforced before figures
Figure artifacts	generated figure registry, manuscript labels/cross-references, PNG information content, captions, alt text, and dashboard figure artifacts agree	enforced by `run_kernels.py` and `validate_manuscript_artifacts.py`
Image pinning	`uv run python scripts/validate_docker_digests.py`	every Compose image reference carries a sha256 digest; the file is parsed, no daemon is contacted
Release inventory	`uv run python scripts/validate_release_readiness_inventory.py`	checked against live git status, untracked files, ignored outputs and blocker probes
Manuscript render	parent template renderer	citation and figure references must resolve

Verification gates for code, prose, artifacts, and rendered surfaces I

Verification gates for code, prose, artifacts, and rendered surfaces I

The second family is live. Every row below needs a reachable Docker daemon, and the browser rows additionally need the `chrome-devtools-axi` Chrome driver. These gates fail closed rather than downgrading to the artifact-only dashboard, which is why they are listed separately instead of folded into the table above.

Verification gates for code, prose, artifacts, and rendered surfaces I

Live gate	Command	Evidence
Live dependency parity	`uv run python scripts/reconcile_live_dependencies.py --check-only`	live prerequisite edges compared with the exported project JSON without writing edges
Live dashboard wiring	`uv run python scripts/validate_live_skilltree_dashboard.py --require-live-admin --require-live-dependencies --require-figure-contracts`	live admin skill-count parity, exact live dependency parity, and figure-contract agreement
Raw SkillTree login	`uv run python scripts/validate_skilltree_form_login.py`	a real browser drives the `FORM` login through to the administrator surface; the JSON summary is redacted
Dashboard console	`uv run python scripts/validate_dashboard_console.py --require-browser --require-dashboard`	no console errors except the allow-listed upstream `Inception` status probe
App-ready stack	`uv run python scripts/validate_full_app_stack.py --app-ready --skip-regenerate`	Compose bring-up, seed, dashboard, login, and console pass; the JSON summary is labeled app-ready
Full-parity stack	`uv run python scripts/validate_full_app_stack.py --skip-regenerate`	the same sequence plus complete live dependency reconciliation; the JSON summary is labeled full-parity

Coverage floor caveat I

The coverage floor is enforced on the aggregate, and the aggregate is not the same claim as every module meeting it. Four modules currently sit below the floor while the suite as a whole clears it: `src/integration/full_app_stack.py`, `src/skilltree/form_login_validation.py`, `src/scholarship/source_health_network.py`, and `src/dashboard/payload.py`. Each one is a boundary to something the offline suite cannot reach — a Docker daemon, a browser subprocess, live HTTP, or the local-auth server path — and each is exercised instead by the live gates in the second table. Two defects show why that exception is a routing decision rather than an excuse: a dashboard content-security-policy directive that blanked the official SkillTree panel was invisible to every offline test and surfaced only in the browser console gate, and a reconciler abort-on-slow-write surfaced

Coverage floor caveat I

only during a complete live dependency run. Read the aggregate number as a property of the suite, not as evidence of uniform module coverage.

Release-evidence ledger and publication blockers

Release inventory comparison I

Release evidence is recorded as a local, machine-checked ledger rather than as a clean public-release assertion. During review, the checkout may intentionally contain dirty or untracked files; `docs/release_readiness_inventory.md` is the authoritative inventory for that state and is regenerated with `uv run python scripts/validate_release_readiness_inventory.py --write`.

Release inventory comparison I

The default validator then compares the document with live `git status --short`, `git ls-files -o --exclude-standard`, `git diff --stat`, ignored generated-output examples, and the current external blocker probes. Those probes report three distinct positions, and collapsing them into one blocked-or-ready summary would misstate all three.

`docker info` reaches the Docker/Colima daemon, so the Docker-dependent live rehearsal is no longer blocked. It has been run: `validate_full_app_stack.py --skip-regenerate` completed against the pinned SkillTree image in full-parity mode, with the live dependency graph matching the export edge for edge, the live admin skill count matching the exported catalog, every registered artifact and figure contract agreeing, the raw `FORM` login reaching the administrator surface, and the browser console gate clean. What remains

is to rerun that sequence as a release handoff rather than as a development rehearsal, on the commit that is actually released.

`git ls-remote` against the public `ActiveInferenceInstitute/Active_Skillference` target now succeeds, and that is precisely why the validator distinguishes reachability from publication: the target exists and holds no refs. An exit-code-only check would read that empty placeholder as ready. The inventory therefore reports it as reachable but empty, and DOI, archive-deposit, and public-repository metadata fields stay blank until refs actually land there. Nothing in this repository is published at that address today.

Release inventory comparison I

The independent cross-vendor audit is closed in `tasks.yaml`, not pending. Its recorded scope is a point-in-time, read-only static review of the dashboard, local-auth, and full-stack sources and their tests, and within that scope it returned no unresolved critical or high findings. It did not cover dependency and supply-chain exposure, secrets in git history, or dynamic runtime testing, so it closes a specific review task rather than establishing operational assurance. Branch hygiene remains open while the checkout still carries modified tracked paths.

Release inventory comparison I

Reproducible-workflow guidance motivates the rerunnable command ledger, versioned code and transparent artifact state (Sandve et al. 2013; Wilson et al. 2014), while FAIR and transparency-oriented open-science standards motivate the metadata and evidence-ledger shape (Wilkinson et al. 2016; Nosek et al. 2015; Munafò et al. 2017). These sources do not turn a dirty local checkout into a public archive. Software-citation and machine-readable metadata guidance define the repository-level citation files and release metadata surfaces (Smith et al. 2016; Citation File Format Project 2026; CodeMeta Project 2026), while GitHub and Zenodo documentation keep DOI language tied to a real public target and deposit (GitHub Docs 2026; Zenodo 2026). SPDX, OSI and REUSE references bound the MIT and third-party-notice signposting (SPDX Workgroup 2026; Open Source Initiative 2026; Free Software Foundation Europe 2024); they do

not provide legal advice, relicense third-party materials, or remove the need for separate deployment review. A release record should include the base commit hash, whether the worktree was dirty, the generated inventory, the commands in the gate table above, and the artifact manifests in `output/data/` and `output/figures/figure_registry.json`; it should not promote this local evidence into a DOI or public-repository claim until the external blockers are resolved.

The release inventory also carries a machine-checked **Key Artifact Hashes** section. During validation, it records SHA-256 digests and byte sizes for `output/data/claims.json`, `output/data/manuscript_variables.json`, `output/data/skilltree_project.json`, `output/figures/figure_registry.json`, and every registered figure PNG currently named by the figure registry. The same validator rejects stale hash rows, so a regenerated artifact cannot silently drift from the manuscript's local reproducibility evidence. This is still local evidence: without a release tag, public archive, and clean external blocker probes, it remains a checkout ledger rather than an immutable publication bundle.

Provenance chain from kernel function to learner-facing value

Provenance chain from kernel function to learner-facing value I

The chain from kernel to learner is auditable end to end: a kernel function computes a value -> `examples.py` registers it as a named `Claim` with its source function -> a skill or manuscript section references it as `claim:NAME` -> the exporter or render hook resolves it, failing closed if it cannot -> the value appears in the SkillTree skill description, the manuscript, and `output/data/claims.json`. Changing a kernel changes the claim, the rendered content, and the figures together. The separate manuscript artifact audit closes the non-numeric drift path: a figure cannot be registered without a manuscript declaration, referenced without a declaration, missing from disk, or absent from the dashboard figure inventory.

Disposable generated artifacts and output inventory

Disposable generated artifacts and output inventory I

scripts/run_kernels.py writes output/data/claims.json (the provenance ledger), output/data/manuscript_variables.json (claims plus curriculum counts), output/data/scholarship_coverage.json (the citation lane audit), output/data/scholarship_gap_report.json (the scholarship frontier-source and claim-boundary audit), output/data/manuscript_artifacts.json (the figure-surface audit), output/data/cover_asset_audit.json (the first-page cover audit), output/data/front_matter_visual_audit.json (the page-two atlas audit), output/data/course_material_index.json (local modular lesson records), and output/data/course_material_audit.json (the lesson-record completeness audit), plus 21 registered figures plus the generated cover and named-skill atlas PNGs under output/figures/. scripts/export_skilltree.py writes

Disposable generated artifacts and output inventory I

`output/data/skilltree_project.json`, the import-ready SkillTree project. `scripts/z_generate_manuscript_variables.py` is the parent-template render hook: it runs the kernel/provenance gate and writes resolved manuscript copies under `output/manuscript/` before PDF, HTML, slide, DOCX, or EPUB rendering.